

AI Interview Coach — Professional Presentation Guide

Use this file as your speaking script and demo checklist. Do not read it word-for-word; use it to stay structured and confident.

1. Presentation goals (what you want the audience to remember)

By the end, listeners should understand:

- 1. **Problem:** Generic interview practice ignores the candidate’s resume and the target job.
- 2. **Solution:** A multi-agent system that personalizes questions from resume + JD, runs a voice interview, then evaluates the full session.
- 3. **Engineering value:** Clear agent roles, orchestration, structured UI, user accounts, document library, conversation history, and downloadable feedback/study packs.
- 4. **Demo proof:** You can show a real end-to-end flow live.

One-line pitch (memorize this):

“AI Interview Coach is a multi-agent mock interview system that turns a candidate’s resume and a job description into a personalized, voice-based practice session with feedback and a reusable prep pack.”

2. Recommended timing (10–12 minutes)

Minutes	Section	Focus
0:00–1:00	Hook + problem	Why generic practice fails
1:00–2:30	Goal + architecture	Multi-agent design (simple diagram talk)
2:30–7:30	Live demo	Login → library → interview → feedback → downloads → history
7:30–9:00	Engineering highlights	Patterns, modularity, persistence
9:00–10:30	Limits + next steps	Honest scope + roadmap
10:30–12:00	Q&A	Short, confident answers

If you only have **5 minutes**: problem (30s) → architecture (60s) → demo highlight reel (3 min) → one limitation + one next step (30s).

3. Slide outline (8 slides max)

You can present from slides or from the running app. If using slides, keep them sparse.

1. **Title** — AI Interview Coach · Your name · Course/project
 2. **Problem** — Generic questions ≠ real interviews
 3. **Solution** — Personalized multi-agent interview coach
 4. **Architecture** — Analysis → Interviewer → Manager → Evaluation (+ UI / library / reports)
 5. **Demo flow** — numbered steps (no screenshots overload)
 6. **Key features** — voice, auth, library, history, downloads
 7. **Tech stack** — Python, Gradio, OpenAI, Agents, local user storage
 8. **What's next** — adaptive follow-ups, cloud storage, stronger evaluation rubric
-

4. Architecture talking points (keep it simple)

Speak in this order:

1. **Job Analysis Agent** — reads resume + JD, finds matches/gaps, generates tailored questions.
2. **Interview Agent** — runs the question sequence (opening → tailored → candidate's question).
3. **Interview Manager** — owns session state, turn order, and handoff between steps.
4. **Evaluation Agent** — scores the full interview after the last answer.
5. **Product layer around agents**
 - Auth (email/password, hashed)
 - Document library (reuse resumes/JDs)
 - Conversation history (read + replay audio)
 - Reports (feedback + study pack as PDF/DOCX)

Phrase that sounds professional:

"The LLM does the language work; the manager agent and application code own the workflow, state, and safety of the experience."

5. Live demo script (follow this exact path)

Before you present (checklist)

- ☐ App starts cleanly (`python app.py`)
- ☐ `.env` has a valid `OPENAI_API_KEY`
- ☐ Sample files ready: `sample_docs/Resume_Java_Developer.pdf` and `sample_docs/JD.docx`
- ☐ Test account ready (or create one live in 10 seconds)
- ☐ Microphone permission allowed in the browser
- ☐ Close unrelated tabs; zoom browser to ~110% so UI is readable
- ☐ Optional: `share=True` only if you need a public Gradio link for remote audience

Demo steps (speak while clicking)

1. Login / Register

- "Users get an account so their documents and past interviews stay with them."

2. Library

- Upload resume + JD once (or select saved ones).
- Point to **Auto-save uploads**.
- "No need to re-upload every practice session."

3. Start interview

- Click **Start interview**.
- Show the first question + audio.
- "Questions are grounded in this resume and this job description."

4. Answer loop (2–3 questions is enough live)

- Record a short answer → **Transcribe** → **Save & continue**.
- "Each answer is stored with its question so evaluation uses the full conversation."

5. Completion (or jump to a saved past interview if time is short)

- Show final feedback.
- Download **feedback** and/or **study pack** (PDF or DOCX).
- "The study pack includes expected questions, strong answers, and concept explanations."

6. Past interviews

- Open a saved conversation named like **JD – 2026-10-09 19:13**.
- Play a question or answer audio clip.
- "Practice history is reviewable, not disposable."

If the live demo fails

Have a backup plan:

1. Open a **past interview** already saved and narrate the flow.
2. Show downloadable report files from **downloads/** if available.
3. Walk the architecture verbally using the agent table.

Never apologize repeatedly — switch to backup in one sentence:

"I'll show a completed session from history so we stay on time."

6. What to emphasize as engineering (not just UI)

Use 3–4 of these; don't dump everything:

- **Multi-agent separation of concerns** (analysis vs interview vs evaluation vs orchestration)
 - **Deterministic interview pipeline** managed in code (predictable 6-question flow)
 - **Speech in/out** (STT + TTS) for a realistic practice loop
 - **Modular codebase** (**agents/**, **auth/**, **library/**, **reports/**, **ui/**)
 - **Per-user persistence** under **user_data/** (documents + conversations + audio)
 - **Downloadable coaching artifacts** (feedback report + study pack)
-

7. Honest limitations (builds trust)

Say these calmly:

- Opening and closing questions are fixed; middle questions are tailored.
- Final evaluation is generated after all answers (not deep adaptive follow-ups each turn).
- Auth and file storage are local/app-level (good for a project demo; not enterprise SSO/S3 yet).
- Model quality depends on input document quality and API availability.

Then immediately add upside:

“Those constraints kept the workflow reliable for a classroom/project demo and make the next improvements obvious.”

8. Strong closing

Close with:

1. Restate the pitch (one sentence).
2. One business/user value: “candidates practice on *their* role, not random questions.”
3. One technical value: “multi-agent orchestration with persistent practice history.”
4. Ask for questions.

Closing line:

“This project shows how agentic patterns become a usable product: personalized interview practice, measurable feedback, and prep materials the candidate can take away.”

9. Likely Q&A (prepare short answers)

Q: Why multi-agent instead of one big prompt?

A: Specialization improves quality and control. Analysis, interviewing, and evaluation are different jobs; the manager coordinates them.

Q: How are questions personalized?

A: The analysis agent compares resume and JD, then generates role-specific questions used in the session.

Q: Where is user data stored?

A: Locally per account under `user_data/` — resumes, JDs, conversation transcripts, and audio clips.

Q: Can users practice again without uploading?

A: Yes — saved documents and past interviews are reusable from the library UI.

Q: What’s next if you had two more weeks?

A: Adaptive follow-up questions, richer scoring rubric, cloud storage, and stronger security for production auth.

10. Delivery tips

- Stand/sit with the app fullscreen; narrate before each click.
 - Keep answers in the demo short (10–20 seconds) so the flow stays moving.
 - Use the status bar text as proof the system is advancing (Q2/6, complete, etc.).
 - Don't open code unless asked; offer "I can show the agent modules if useful."
 - If asked for code structure, show folders only: `agents/`, `auth/`, `library/`, `reports/`, `ui/`.
-

11. Quick run commands (for setup slide or appendix)

```
cd project1/Agentic-Ai-project1-Interview-assistance-agent-  
source ../../.venv/bin/activate    # or your environment  
python app.py
```

Open the local Gradio URL (usually <http://localhost:7860>).

12. 30-second version (elevator)

"People practice interviews with generic questions that don't match their background or the job. AI Interview Coach uses multiple agents to analyze a resume and job description, run a voice mock interview, evaluate the full session, and generate downloadable feedback plus a study pack. Users can save documents and revisit past conversations. It's a practical example of agentic workflow design turned into a real product experience."